

Creating Factories

Now that we saw how the user factory works, let's create a factory for the **Job** model.

Creating a Factory

To create a factory for the **Job** model, run the following command:

```
php artisan make:factory JobFactory
```

This will create a file at `database/factories/JobFactory.php`. It will create a class called **JobFactory** that extends the **Factory** class with a method called **definition**. The **definition** method returns an array of attributes that will be used to create a new **Job** model. Same as with the user factory that we looked at in the last lesson.

We need to have a user for each listing, so let's bring in the **User** model:

```
use App\Models\User;
```

Now, let's define the **definition** method:

```
public function definition(): array
{
    return [
        'user_id' => User::factory(), // Create new user for each listing
        'title' => $this->faker->jobTitle,
        'description' => $this->faker->paragraphs(3, true),
        'salary' => $this->faker->numberBetween(40000, 120000),
        'tags' => implode(', ', $this->faker->words(3)),
        'job_type' => $this->faker->randomElement(['Full-Time', 'Part-Time',
'Contract', 'Temporary', 'Internship', 'Volunteer', 'On-Call']),
        'remote' => $this->faker->boolean,
        'requirements' => $this->faker->sentences(3, true),
        'benefits' => $this->faker->sentences(2, true),
        'address' => $this->faker->streetAddress,
        'city' => $this->faker->city,
        'state' => $this->faker->state,
        'zipcode' => $this->faker->postcode,
        'contact_email' => $this->faker->safeEmail,
        'contact_phone' => $this->faker->phoneNumber,
        'company_name' => $this->faker->company,
        'company_description' => $this->faker->paragraphs(2, true),
        'company_logo' => $this->faker->imageUrl(100, 100, 'business', true,
'logo'),
        'company_website' => $this->faker->url,
```

```
    ];  
}
```

Here is an explanation of each attribute:

- **user_id**: Creates a new user for each job listing using the - `User::factory()` method.
- **title**: Generates a random job title.
- **description**: Creates a description with three paragraphs.
- **salary**: Random salary between 40,000 and 120,000.
- **tags**: Generates a string of three random words separated by commas.
- **job_type**: Selects a random job type from the predefined list.
- **remote**: Randomly determines if the job is remote or not.
- **requirements**: Generates a string of three sentences for job requirements.
- **benefits**: Generates a string of two sentences for job benefits.
- **address**: Generates a random street address.
- **city**: Generates a random city name.
- **state**: Generates a random state name.
- **zipcode**: Generates a random postal code.
- **contact_email**: Generates a safe email address.
- **contact_phone**: Generates a random phone number.
- **company_name**: Generates a company name.
- **company_description**: Creates a company description with two paragraphs.
- **company_logo**: Generates a URL to a random image (representing a company logo).
- **company_website**: Generates a random URL for the company's website.

Now that we have a factory for the **Job** model, let's use it to create some jobs.

Run The Job Factory

To create a job listing using the factory, open Tinker by running `php artisan tinker` and run the following command:

```
\App\Models\Job::factory()->create();
```

Let's create 10 more by using the **count** method:

```
\App\Models\Job::factory()->count(10)->create();
```

Yes sir! You should have 11 jobs in your database. You can check with the following command:

```
\App\Models\Job::all();
```

You can also go to your app and you should see the titles of the jobs on the `/jobs` page.

How cool is that? All this right out of the box. Imagine how much time you would have saved if you had to create all these jobs manually. This is the power of Laravel and its ecosystem.